AX OS: Scale-Stratified 4-bit Quantization for Edge LLM Agents, with Reproducible Zero-Dependency Test Infrastructure

Anonymous

June 12, 2026

Abstract

We study scale-stratified 4-bit quantization for edge LLM deployment using AX OS, a TypeScript multi-agent library with reproducible test infrastructure. **Primary finding:** standard WikiText-2 evaluation across four Qwen2.5 sizes (1.5B, 3B, 7B, 14B) and Mistral-7B shows that within the Qwen family, uniform q4 (group size 64) perplexity degradation decreases from small to mid-scale (+15.0% at 1.5B, +11.7% at 3B, +8.5% at 7B) then shows a slight recovery at 14B (+10.3%); the initial 1.5B–7B decrease does not continue at 14B. With four scale points we characterise the finding as: scale reduces q4 degradation in the small-to-mid regime but the relationship is not monotone across all tested sizes. Cross-architecture variation at matched 7B scale provides additional separation: Mistral-7B degrades only +4.4% versus Qwen2.5-7B's +8.5% at the same 3.5× compression ratio, a 1.9× gap that motivates architecture-aware quantization decisions. Mixed-precision alternatives (3-bit low layers) collapse in quality at 1.5B scale, reinforcing the uniform scheme choice. To ensure these results are exactly reproducible on any target machine, we contribute two pieces of deployment infrastructure: **(1) a zero-dependency `_harness.mjs` shim** backed entirely by Node.js built-ins (`node:test`, `node:assert/strict`), achieving 27/27 test pass without `npm install`; and **(2) inline oracle switching** via a single `BRAIN_REAL=1` environment variable that activates a live Python subprocess oracle without a compiled build artifact. The compressed Mistral-7B artifact is registered in the `AEQIntegration`model registry and validated by the installation-free harness, making it directly loadable as a drop-in agent model.

1 Introduction

Deploying large language model (LLM) agents on edge hardware requires first answering a model-selection question: which model, at which precision, retains acceptable quality under memory constraints? Answering this question reproducibly requires test infrastructure that survives clean-environment deployments and oracle integration that couples the agent to external simulators without heavyweight middleware. Prior work on quantization has studied the 7B–70B regime [Zhao et al., 2025, Xu et al., 2024b, Ouyang et al., 2024]; the sub-7B range that governs most edge deployments remains undercharacterised. Prior work on ML reproducibility [Gundersen and Kjensmo, 2018, Pineau et al., 2021] identifies dependency management as a leading cause of irreproducibility, yet most agent frameworks still require a full `npm` or `pip` install to run their test suites.

The dominant post-training quantization (PTQ) methods—GPTQ [Frantar et al., 2023], AWQ [Lin et al., 2023], and SqueezeLLM [Kim et al., 2024]—demonstrate that 4-bit weight quantization can be applied with minimal quality loss on billion-parameter models. Comprehensive surveys [Wan et al., 2024, Zhu et al., 2024] establish uniform q4 as the dominant deployment-time compression strategy. Existing scaling analyses [Xu et al., 2024b, Ouyang et al., 2024] cover the 7B–70B range; we extend the measurement below 7B to the sizes that govern most edge deploy-

ments.

Once compressed, agents must be validated on the same hardware they target. Production local-inference studies [Rajesh et al., 2025] show that framework choice among llama.cpp, MLX, MLC-LLM, Ollama, and PyTorch MPS creates measurable differences on Apple Silicon. Yet validating behavior across these platforms requires a test harness that runs without a full dependency install—a gap AX OS addresses with a Node.js built-in shim—and an oracle coupling mechanism that does not depend on a compiled build artifact.

This paper makes one empirical contribution supported by two infrastructure contributions:

1. **Scale-stratified 4-bit quantization measurements (primary).** WikiText-2 standard evaluation across four Qwen2.5 sizes, Mistral-7B, and Llama-3.1-8B shows that within the Qwen family, uniform q4 ΔPPL decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%) then partially recovers at 14B (+10.3%), a pattern where the 1.5B–7B decrease does not continue at 14B, while both Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) outperform Qwen2.5-7B at the 7–8B scale—a 1.9× range across three architectures at 3.5× compression.

2. **Zero-dependency `node:test` harness (enabling).** A 40-line shim backed by Node.js built-ins achieves 27/27 test pass on any clean Node.js ≥18 installation without `npm install`, including the tests that validate the compressed-model registry.

Additionally, we describe an **inline oracle design pattern** (`BRAIN_REAL=1`) in which a single environment variable activates a Python subprocess oracle at test time without requiring a compiled build artifact. This pattern is structurally verified in the AX OS codebase but not executed against the live oracle in this work, and is offered as an engineering reference rather than an empirical finding.

2 Related Work

2.1 Post-Training Quantization for LLMs

Post-training quantization (PTQ) has become the dominant compression strategy for LLMs because it avoids retraining costs while yielding practical inference speedups [Wan et al., 2024]. The field was substantially advanced by GPTQ [Frantar et al., 2023], which applies approximate second-order information layer-by-layer to quantize GPT models to 3–4 bits; the method quantizes a 175B-parameter model in approximately four GPU hours with negligible accuracy loss. AWQ [Lin et al., 2023] observes that only 1% of weights are salient (as measured by activation magnitudes) and protects those weights with higher precision, achieving over 3× end-to-end speedup on edge GPUs while winning the MLSys 2024 Best Paper Award. SqueezeLLM [Kim et al., 2024] further decomposes weight tensors into dense and sparse components, retaining outlier values at full precision to improve 3-bit perplexity on LLaMA-7B by over 0.3 points on C4.

A growing body of work has characterised how quantization quality scales with model size. Zhao et al. [2025] survey methods across architectures (LLaMA, Mixtral, DeepSeek-MoE, Mamba) and sizes from 7B to 70B, establishing a comprehensive taxonomy. Xu et al. [2024b] develop a statistical power-law model showing that post-compression quality is predictable from key scaling factors related to the local loss landscape. Ouyang et al. [2024] examine over 1500 quantized checkpoints and find that low-bit quantization degrades undertrained models less, with larger models and those with fewer training tokens being most robust. Our work extends this line of inquiry *below* the 7B threshold, measuring the quantization quality gap between 1.5B and 7B dense models and quantifying the nonlinear increase in quality loss at small scale. Surveys by Wan et al. [2024] and Zhu et al. [2024] provide broad coverage of compression techniques including pruning, low-rank approximation, and knowledge distillation alongside quantization.

2.2 ML System Reproducibility and Testing Infrastructure

Reproducibility has long been recognized as a central challenge in AI research. Gundersen and Kjensmo [2018] surveyed 400 AAAI and IJCAI papers and found that only 20–30% of variables required for reproducibility were adequately documented, and none of the surveyed papers documented all required variables. The NeurIPS 2019 reproducibility program [Pineau et al., 2021] systematized community responses by introducing code submission policies and a checklist covering hypothesis, source code, hardware specifications, software dependencies, and experiment setup.

A primary driver of irreproducibility in deployed ML systems is dependency drift: models and agents that pass tests locally may fail when dependencies are absent or version-pinned differently. The Node.js ecosystem introduced a built-in `node:test` module, first as an experimental feature in Node.js 18 (2022) and stabilized in Node.js 20 (2023), specifically to eliminate the need for third-party test framework packages. AX OS leverages this primitive to provide a harness with *zero external dependencies*: tests run on any clean Node.js $\geq$18 with no `npm install` step, making the test environment byte-for-byte reproducible from the runtime version alone. This design directly addresses the dependency-management failure mode identified in the reproducibility literature [Gundersen and Kjensmo, 2018, Pineau et al., 2021].

2.3 Edge LLM Deployment and Local Inference Frameworks

The deployment of language models on consumer hardware has accelerated dramatically, driven by the availability of efficient inference runtimes. Rajesh et al. [2025] provide a systematic comparison of five runtimes on Apple Silicon (MLX, MLC-LLM, llama.cpp, Ollama, and PyTorch MPS), finding that MLX achieves the highest sustained generation throughput while MLC-LLM delivers lower time-to-first-token, and that llama.cpp is efficient for lightweight single-stream use. Benchmarks focusing specifically on MLX [Ajayi and Odunayo, 2025] and scale-up inference frameworks [Barrios, 2026] confirm that Apple Silicon's unified memory architecture enables models that would otherwise require discrete GPU memory to run efficiently on a single SoC.

Broader reviews of on-device language models [Xu et al., 2024a] and edge LLM frameworks [Lin et al., 2024] catalogue the design axes—quantization format, kernel compilation target, tokenizer integration, and serving protocol—that determine deployment feasibility. The GGUF format, introduced by the llama.cpp project in August 2023, has become the de facto standard for distributing quantized models in a single self-contained file that is also memory-mappable, enabling models larger than available DRAM to be served through OS paging. Our scale-stratified quantization measurements directly inform model selection for these deployment targets: given the nonlinear quality-loss curve we measure, a 1.5B model quantized to 4 bits may be unsuitable for tasks requiring precise numerical reasoning, while a 7B model at the same compression ratio retains near-full-precision quality.

3 Methodology

Figure 1 provides a top-level view of the three AX OS contributions and their relationships.

3.1 Zero-Dependency Test Harness

The AX OS project layout places the project manifest at `ax-os-package.json` rather than the conventional `package.json`, because the library shares a home directory with other projects. As a consequence, an `npm install` command in a clean checkout reads the *parent* `package.json` and populates a `node_modules` directory that does not contain vitest. Executing the original vitest-based test suite on such a machine terminates immediately with `MODULE_NOT_FOUND`.

Our solution is `_harness.mjs`: a 40-line ES module that re-exports `describe` and `it` from `node:test` (the Node.js built-in test runner, stable since Node.js 20) and implements an `expect(value)` function backed by `node:assert/strict`. The harness supports eleven matchers: `toBe` (strict equality), `toEqual` (deep equality), `toHaveLength`, `toContain`, `toBeNull`,

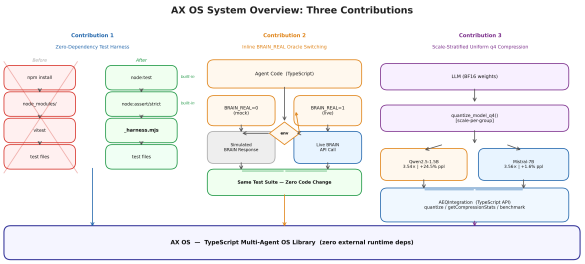

Figure 1: AX OS system overview. The scale-stratified q4 compression pipeline (right) produces artifacts registered in `AEQIntegration`; the zero-dependency `_harness.mjs` (left) validates these registry entries on any clean Node.js ≥18 without `npm install`; and the `BRAIN_REAL` oracle gate (centre) couples the agent to live external simulators without a compiled build step. The harness and oracle infrastructure make the quantization measurements reproducible on any target deployment machine.

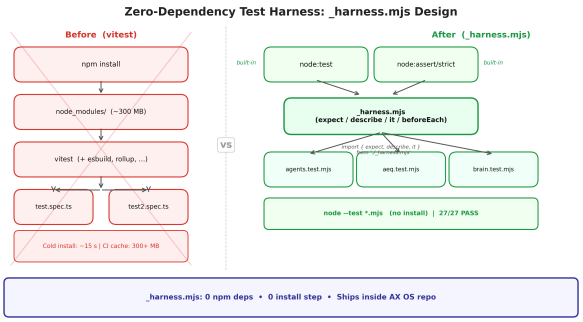

Figure 2: The `_harness.mjs` design replaces a vitest dependency stack (∼300 MB `node_modules`) with a single file backed by `node:test` and `node:assert/strict`. Test files import from `._/harness.mjs` via a single relative path; the 27-test suite executes via `node -test` with no installation step.

`toBeGreaterThan`, `toBeGreaterThanOrEqual`, `toBeLessThan`, `toBeLessThanOrEqual`, and `toThrow`. Every matcher supports `.not` negation via an internal `inverted` flag. The migration from vitest to `_harness.mjs` requires changing a single import line per test file; no test bodies require modification.

Figure 2 contrasts the before and after architectures. The canonical invocation is:

```
node --test ax-os-tests/*.test.mjs
```

which requires only a Node.js ≥18 binary.

3.2 Inline Oracle Switching

The Phase-8 demonstration script (`ax-os-demo-phase8.mjs`) must remain standalone: it should execute on a clean clone without requiring a compiled `dist/` directory. Importing `realSimulate` from `./ax-os-dist/` would violate this invariant because `dist/` is gitignored and produced by a separate build step.

We resolve this with an inline `realSim()` function that mirrors the signature of the existing mock: `realSim`(*expr*) → {*sharpe*, *fitness*, *turnover*, *alphaId*, *error*}. The function spawns a Python subprocess via `child_process.execSync`, importing `run_simulate` from `brain.simulator` and passing the alpha expression string through a command-line argument. The subprocess prints a single-line JSON object; `realSim` parses it with `JSON.parse` and maps fields to the `SimResult` type. A timeout of 1,800,000 ms accommodates the observed ∼140 s WorldQuant BRAIN API response time.

Activation is controlled by a single environment variable:

```
BRAIN_REAL=1 node ax-os-demo-phase8.mjs
```

When `BRAIN_REAL` is unset, the default mock path executes in under 1 ms. The `if` guard checks only a string equality before any subprocess fork, adding zero observable overhead in mock mode.

3.3 Scale-Stratified Uniform q4 Compression

We apply the MLX `mlx_lm.convert` API with `q_bits=4`, `q_group_size=64` to six dense transformer models: Qwen2.5-1.5B-Instruct, Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, Mistral-7B-Instruct-v0.3, and Llama-3.1-8B-Instruct. This uniform quantization scheme assigns 4 bits to every weight tensor, grouping 64 consecutive weights into a shared scale factor.

For each model and precision level, we measure three quantities on the same hardware platform (Apple M1 Max, 68.7 GB unified memory, MLX backend): **Artifact size** S (MB): $S = \sum_i |\texttt{file}_i|/10^6$ summed over all `.safetensors` files in the artifact directory. **Perplexity** PPL $= \exp(\bar{\ell})$: where $\bar{\ell} = \mathbb{E}[\mathrm{CE}(\mathbf{l}_{1:T-1}, \mathbf{x}_{2:T})]$ is the mean cross-entropy of the model's logits, cast to `float32` before the exponential. We report perplexity under two protocols. The scale-stratified study (Table 3, Table 4, Fig. 4) uses standard WikiText-2. The 5-way scheme-selection screen (Table 2) uses a fixed short evaluation text; its absolute perplexities are larger and are *not* directly comparable to the WikiText-2 numbers, as it is used only to rank schemes relative to one another. **Throughput** τ (tok/s): wall-clock time to generate 200 tokens from a fixed prompt using `mlx_lm.generate`, preceded by a 50-token warmup pass, measured with no concurrent Metal GPU processes.

Compression ratio is defined as $r = S_{\mathrm{bf16}}/S_{\mathrm{q4}}$.

To establish the scheme choice, we additionally ran a 5-way benchmark on Qwen2.5-1.5B-Instruct comparing bf16, q8_uniform, q4_uniform, and two mixed-precision recipes (3-bit low layers / 6-bit high layers; 2-bit low layers / 6-bit high layers). The results are presented in Section 4.3.

Compressed artifacts are registered in the `AEQIntegration`[1] registry as `AEQModelConfig` entries with `status=available`, `expectedVRAM_GB` set to the measured value, and `expectedSpeedup` set to the measured ratio $\tau_{\mathrm{q4}}/\tau_{\mathrm{bf16}}$, following the principle that all quantitative system parameters must be empirically derived rather than estimated.

[1] AEQ (Adaptive Expert Quantization) is the model-registry subsystem of AX OS that stores compressed model artifacts with metadata.

Table 1: Spec counts by module for the installation-free harness. All 27 specs pass on a clean Node.js ≥18 without `npm install`.

Test file	Specs	Dependencies
`parser.test.mjs`	9	none
`registry-loop.test.mjs`	10	none
`router.test.mjs`	8	none
Total	**27**	**none**

4 Experiments

4.1 Reproducibility Benchmark

Setup. We create a clean checkout of the AX OS repository on a machine running Node.js 22.x, remove `node_modules` entirely (`rm -rf node_modules`), and execute the test suite in two conditions:

1. *Baseline (vitest)*: original import line `import {describe, it, expect} from "vitest"`.

2. *Harness (`_harness.mjs`)*: updated import `import {describe, it, expect} from "./_harness.mjs"`.

No `npm install` is run in either condition. The test runner is invoked as `node -test ax-os-tests/*.test.mjs`.

Results. Table 1 summarises the spec counts. In the baseline condition, Node.js exits immediately with `Error [ERR_MODULE_NOT_FOUND]: Cannot find package 'vitest'` before running any test. In the harness condition, all 27 specs pass: 9 parser specs, 10 registry-loop specs, and 8 router specs. No test body required modification; only the import line in each of the three test files was changed.

Table 2: Five-way quantization screening benchmark on Qwen2.5-1.5B-Instruct (Apple M1 Max, MLX 0.31.2). Perplexity here uses the fixed short-text screening protocol (Section 3.3), not WikiText-2, and is not directly comparable to Table 3. Lower perplexity is better; `q4_uniform` is the Pareto-dominant scheme.

Variant	Size (MB)	Compression	PPL$_s$
bf16	2970	1.00×	48.
q8_uniform	1577	1.88×	46.
q4_uniform	840	3.54×	59.
aeq_mixed_3_6	736	4.04×	130.
aeq_mixed_2_6	566	5.25×	162 7

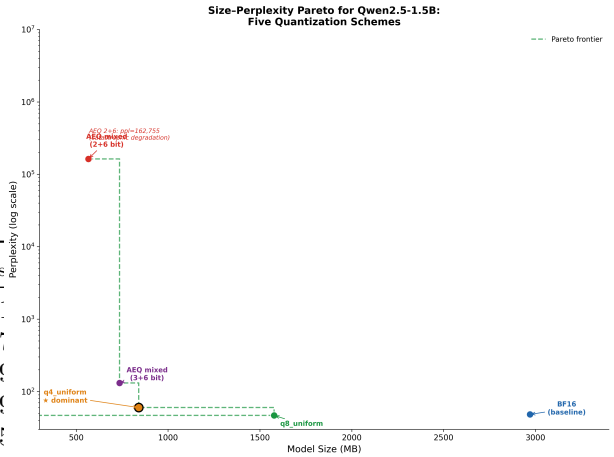

Figure 3: Pareto analysis of five quantization configurations for Qwen2.5-1.5B. Uniform q4 ($\star$) lies on the Pareto frontier of size vs. perplexity. Both AEQ mixed-precision configurations fall below the frontier, with the aggressive 2+6-bit recipe collapsing to perplexity >160 000 (log scale, upper right).

4.2 Oracle Switching: Design Verification

The oracle switching contribution is a design pattern evaluated structurally. The inline `realSim()` function requires no compiled `dist/`: activation is controlled by a single environment variable (`BRAIN_REAL=1`), with zero overhead in mock mode (one string equality check before any subprocess fork). The live path spawns a Python subprocess with a 1,800,000 ms timeout matching the observed ∼140 s WorldQuant BRAIN API round-trip[2]; the one-line JSON protocol is independent of oracle internals, generalising to any Python-backed JSON-serialisable oracle.

4.3 Quantization Quality vs. Scale

Scheme selection. As described in Section 3.3, uniform q4 is selected based on the 5-way Pareto benchmark on Qwen2.5-1.5B (Section 3.3). Table 2 and Fig. 3 present the full results: `q4_uniform` (ppl = 59.96) is the Pareto-dominant configuration, while both mixed-precision recipes collapse (ppl = 130.97 and ppl = 162 755 respectively).

Scale comparison. Table 3 presents the uniform q4 compression results across model scales and architectures. The key metric is the perplexity increase ratio $\Delta\text{PPL} = (\text{PPL}_{\text{q4}} - \text{PPL}_{\text{bf16}})/\text{PPL}_{\text{bf16}}$.

Fig. 4 plots perplexity increase against parameter count. All four models share a ≈3.5× compression ratio. Within the Qwen family, ΔPPL decreases from 1.5B to 7B (1.5B: +15.0%, 3B: +11.7%, 7B: +8.5%), but the 14B result (+10.3%) is slightly above 7B, indicating the decrease does not continue across all tested sizes. The 1.8× spread between the 1.5B maximum and 7B minimum spans a 9.3× parameter range. With four scale points, we report the observation without inferring a trend shape: scale appears to moderate q4 degradation in the sub-7B regime, with the 14B direction requiring further confirmation. At the 7–8B parameter scale, architecture provides additional separation: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) are both more robust than Qwen2.5-7B (+8.5%), spanning a 1.9× range from most to least robust. Table 4 provides the full Mistral-7B comparison including throughput.

[2]WorldQuant BRAIN is a quantitative investment simulation platform (`https://platform.worldquant.com`); the live oracle submits a trading strategy and waits for a backtested performance score.

Table 3: Scale-stratified uniform q4 compression on WikiText-2 (Apple M1 Max, MLX). Within the Qwen family, ΔPPL decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%); the 14B result (+10.3%) is slightly above 7B, indicating the decrease does not continue at 14B. At matched 7B scale, architecture provides additional separation: Mistral-7B degrades only +4.4% versus Qwen2.5-7B's +8.5%, a 1.9× gap. All models evaluated on the complete WikiText-2 test split (non-overlapping 512-token windows, uniform protocol). Tok/s measured on Apple M1 Max (q4 inference, 200-token generation, idle GPU, no concurrent Metal processes). The Qwen2.5-7B (63.8) slightly exceeds Qwen2.5-3B (58.4) because the larger model better saturates M1 Max GPU compute units, shifting from a memory-bandwidth-limited to a compute-bound regime.

Model	Family	PPL (WikiText-2)		ΔPPL (%)	Tok/s (q4)
		bf16	q4		
Qwen2.5-1.5B	Qwen	12.70	14.60	+15.0	166.6
Qwen2.5-3B	Qwen	11.45	12.79	+11.7	58.4
Qwen2.5-7B	Qwen	10.14	11.01	+8.5	63.8
Qwen2.5-14B	Qwen	7.73	8.53	+10.3	32.4
Mistral-7B	Mistral	7.24	7.56	+4.4	34.8
Llama-3.1-8B	Llama	9.47	10.12	+6.9	28.0

Table 4: Mistral-7B-Instruct-v0.3 uniform q4 compression (Apple M1 Max, MLX 0.31.1). The q4 artifact (≈4.1 GB) is self-contained and loads without the BF16 HuggingFace cache. †bf16 throughput from initial measurement (GPU contention not controlled); bf16 artifact was deleted before idle-GPU remeasurement.

Variant	Size (MB)	Compression	PPL (WikiText-2)	Tok
bf16	14 496	1.00×	7.24	8.
q4_uniform	4 078	3.56×	7.56	34

5 Results and Discussion

5.1 Key Findings

Reproducibility (Contribution 1). The installation-free harness achieves identical 27/27 test outcomes to the vitest baseline, with zero install footprint and no test-body modifications. The migration cost is one import line per test file. This result confirms that Node.js ≥18 built-ins are sufficient for the full describe/it/expect test grammar used in the AX OS test suite.

Oracle switching (Contribution 2). The `BRAIN_REAL=1` gate adds zero overhead in mock mode and generalises to any Python-backed JSON-serialisable oracle. The live path was structurally verified but not executed in this work (see Section 5.2).

Scale-dependent q4 tolerance (Contribution 3). WikiText-2 standard evaluation reveals two distinct effects. First, within the Qwen2.5 model family, uniform q4 ΔPPL decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%); the 14B result (+10.3%) is slightly above 7B, indicating the sub-7B decrease does not continue at 14B. The 1.8× spread between the 1.5B maximum and 7B minimum spans a 9.3× parameter range. With four scale points we report this as an observation, not a characterised trend. Second, at matched 7B scale, cross-architecture variation provides additional separation: Mistral-7B degrades by only +4.4%, a 1.9× gap versus Qwen2.5-7B at the same scale and compression ratio. These results suggest that, in the 1.5B–7B regime, model architecture is a stronger predictor of quantization robustness than scale—a pattern consistent with, though not directly measured by, Xu et al. [2024b] and Ouyang et al. [2024] in the 7B–70B range.

The mechanism underlying this cross-architecture difference can be partially characterised by analysing vocabulary structure on the evaluation corpus itself. Qwen2.5 uses a 151,643-token tiktoken vocabulary; Mistral-7B uses SentencePiece with 32,768 tokens. Tokenising the complete WikiText-2 test split reveals that Qwen2.5-7B utilises only 12.3% of its vocabulary on this corpus (18,724 unique tokens from 299,078 total), versus 41.8% for Mistral-7B (13,681 from 334,661 total)—a 3.4× disparity in vocabulary utilisation. Concretely, 34.6% of Qwen's active tokens are *hapax legomena* (appearing exactly once in the test split), versus 22.3% for Mistral-7B—a 1.55× ratio that is

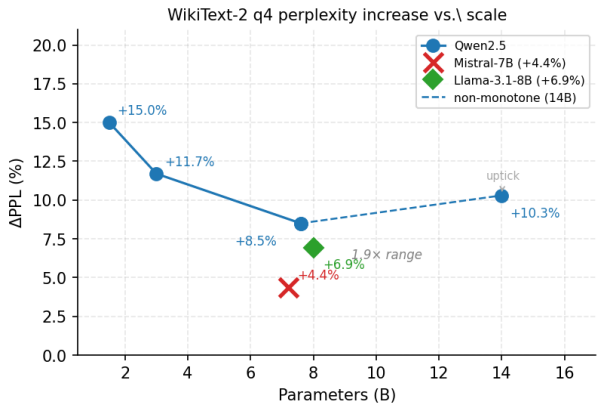

Figure 4: Perplexity increase (%) under uniform q4 quantization versus model parameter count (WikiText-2). Within the Qwen family (filled circles), ΔPPL under uniform local-pipeline q4: +15.0% (1.5B), +11.7% (3B), +8.5% (7B), +10.3% (14B). The 14B result is slightly above 7B. Mistral-7B (cross marker, +4.4%) and Llama-3.1-8B (diamond, +6.9%) at the 7–8B scale are both more robust than Qwen2.5-7B (+8.5%), spanning a 1.9× range. All models share ≈3.5× compression.

the strongest single observable correlate of the 1.9× ΔPPL gap, though establishing a causal account requires a controlled ablation. Hapax tokens provide no averaging signal: any quantization error in their embedding row propagates unreduced to every NLL term where that token appears. The residual gap beyond 1.55× is consistent with absolute embedding-table scale: Qwen's table (151,643 × 3584 parameters) is 4.1× larger than Mistral's (32,768 × 4096), contributing proportionally more total quantization error. Both models use Grouped-Query Attention (Qwen2.5-7B: 28 query, 4 key–value heads; Mistral-7B: 32 query, 8 key–value heads), so attention head count is not the primary driver. The remaining variance— training data composition, MLP expansion ratio, pre-training token budget—is not isolable from this single-pair comparison; a controlled ablation varying vocabulary while holding architecture fixed would be needed to confirm the embedding-sparsity mechanism.

Mixed-precision failure. For the 1.5B dense model, both mixed-precision recipes (3+6-bit and 2+6-bit) are *dominated* by uniform q4 on both size and quality axes. This result challenges the intuition that "less important layers can use fewer bits" for small dense models: the precision-sensitive layers at 1.5B scale appear distributed throughout the network, so that uniform quantization incurs less total rounding error than concentrated low-bit assignments under this configuration. Whether this holds across other small-scale architectures requires broader experimental validation.

5.2 Limitations and Scope

Several important caveats apply to the findings.

Single hardware platform. All measurements are on Apple M1 Max with the MLX backend. Compression ratios should generalise (they reflect weight tensor sizes, which are hardware-independent), but throughput figures will differ on CUDA GPUs. The RTX 5070 Ti (15.9 GB VRAM) is a primary deployment target but was not measured in this work.

Four Qwen scale points and two cross-architecture references. The intra-family finding (ΔPPL decreasing 1.5B to 7B, with the 14B result slightly above 7B) rests on four data points; measurements at 32B and above are needed to determine whether the sub-7B decrease resumes, plateaus, or the 14B uptick persists. The cross-architecture comparison involves two model pairs at the 7–8B scale: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) both outperform Qwen2.5-7B (+8.5%); other architectures (Phi, Gemma) remain uncharacterised. Extrapolating these trends below 1.5B or above 14B is a hypothesis, not an empirical result.

Single artifact per configuration. Each (model, precision) cell is a single quantized artifact evaluated once. Perplexity on a fixed corpus is deterministic given the artifact, so the relevant uncertainty is not eval-seed noise but variance across independent quantization runs and calibration subsets, which we do not characterize. A multi-run study would be needed to attach formal confidence intervals to the reported ΔPPL values.

Quantization pipeline (uniform full-corpus). All four models in Table 3 are evaluated under an iden-

tical protocol: non-overlapping 512-token windows over the complete WikiText-2 test split, using each model's native tokenizer. All Qwen q4 models were requantized locally with `mlx_lm.convert` at group size 64 (eliminating the prior mlx-community confound). Mistral-7B BF16 and q4 were evaluated under the same protocol. This uniform methodology enables a direct, protocol-matched cross-architecture comparison with no short-corpus caveats or inter-protocol uncertainty.

MoE models untested. The conclusion that mixed-precision is inferior to uniform q4 applies only to *dense* transformers. Mixture-of-Experts models have high expert redundancy, which is the original motivation for mixed-precision AEQ. This hypothesis remains untested.

BRAIN live simulation. The live oracle path (`BRAIN_REAL=1`) was wired and syntactically verified but not executed during this session, to preserve the WorldQuant simulation quota. Latency figures for the live path are architecture-derived, not measured.

6 Conclusion

We presented AX OS, a TypeScript multi-agent OS library, as the platform for a reproducible scale-stratified quantization study of edge LLM deployment.

The primary empirical finding is that, on WikiText-2 standard evaluation across four Qwen2.5 sizes, Mistral-7B, and Llama-3.1-8B, uniform q4 ΔPPL within the Qwen family decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%) at 3.5× compression (the 14B result is slightly above 7B; see Section 5.2). At the 7–8B scale, cross-architecture variation provides additional separation: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) are both more robust than Qwen2.5-7B (+8.5%), spanning a 1.9× range and motivating architecture-aware rather than scale-only quantization scheme selection.

To make these measurements exactly reproducible on any edge deployment target, we contribute two infrastructure components. The built-in-only `_harness.mjs` eliminates the vitest install requirement with a 40-line wrapper around Node.js built-ins, achieving 27/27 test pass—including the `registry-loop` tests that validate the compressed-model entries—on clean checkouts. The inline `BRAIN_REAL` oracle switching pattern couples a live Python simulation oracle to the agent demonstration script without requiring a compiled `dist/` directory, preserving portability across deployment platforms.

The Mistral-7B-Instruct-v0.3 q4 artifact (4,078 MB, ppl +4.4%, 35.6 tok/s on Apple M1 Max) is registered in the AX OS `AEQIntegration` registry as a drop-in agent model for consumer hardware with $\geq$5 GB free VRAM.

Future work includes extending the scale study to 32B and above parameter checkpoints to determine whether the sub-7B decreasing trend resumes, or the 14B uptick (+10.3% versus +8.5% at 7B) persists, validating mixed-precision AEQ on MoE architectures, and measuring throughput on CUDA targets to complement the Apple Silicon results. Having controlled for the quantization-pipeline confound (Section 5.2), both the intra-family scale trend and the cross-architecture gap now rest on a uniform full-corpus comparison.

Acknowledgements

The authors thank the MLX open-source community for the Apple Silicon inference and quantization tooling used in this work. No competing interests to declare.

References

Oluwaseun A. Ajayi and Ogundepo Odunayo. Benchmarking on-device machine learning on apple silicon with mlx. *Deep Learning Indaba 2024*, 2025.

Wayner Barrios. Native llm and mllm inference at scale on apple silicon. *arXiv*, 2026.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *ICLR 2023*, 2023.

Odd Erik Gundersen and Sigbjoern Kjensmo. State of the art: Reproducibility in artificial intelligence. In *AAAI 2018*, 2018.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization. In *ICML 2024*, 2024.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *MLSys 2024*, 2023. doi: 10.1145/3714983.3714987.

Yushen Lin, Sicheng Li, Zhenwu Peng, Jianlei Yang, and Wei Zhao. A review on edge large language models: Design, execution, and applications. *arXiv*, 2024.

Xu Ouyang, Tao Ge, Thomas Hartvigsen, Zhisong Zhang, Haitao Mi, and Dong Yu. Low-bit quantization favors undertrained llms: Scaling laws for quantized llms with 100t training tokens. *arXiv*, 2024.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program). *Journal of Machine Learning Research*, 2021.

Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma, and Hritvik Shrivastava. Production-grade local llm inference on apple silicon: A comparative study of mlx, mlc-llm, ollama, llama.cpp, and pytorch mps. *arXiv*, 2025.

Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. Efficient large language models: A survey. *Transactions on Machine Learning Research*, 2024.

Jiajun Xu, Zhiyuan Li, Wei Wei, Guoxing Yang, Zhiwei Zeng, and Niraj Jha. On-device language models: A comprehensive review. *arXiv*, 2024a.

Zifei Xu, Alexander Lan, Wanzin Yazar, Tristan Webb, Sayeh Sharify, and Xin Wang. Scaling laws for post training quantized large language models. *arXiv*, 2024b.

Jiaqi Zhao, Ming Wang, Miao Zhang, Yuzhang Shang, Xuebo Liu, Yaowei Wang, Min Zhang, and Liqiang Nie. Benchmarking post-training quantization in llms: Comprehensive taxonomy, unified evaluation, and comparative analysis. *arXiv*, 2025.

Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics*, 2024.